📑 DOCUMENTO DE ESPECIFICAÇÃO TÉCNICA E ARQUITETURA DE CÓDIGO (VERSÃO V6.1)

Projeto: Nanopaint LIMS & ERP Evolution

Filosofia Arquitetural: Clean Architecture + Offline-First Data-Driven Architecture (Soberania Pragmática)

Tecnologias Core: WinUI 3 (Windows App SDK 1.6+) + .NET 8 LTS + PostgreSQL + SQLite com SQLCipher + Puppeteer Sharp + Handlebars.NET + PdfSharpCore + Microsoft WebView2.

PARTE 1: SISTEMA DE PASTAS E ARQUITETURA DE CÓDIGO

1. Estrutura Genérica da Solução (.sln)

A arquitetura baseia-se na separação estrita de responsabilidades (*Separation of Concerns*). O núcleo da aplicação (**Core**) é totalmente agnóstico a bases de dados e motores de renderização, garantindo que as regras de controlo de qualidade e engenharia de polímeros da Nanopaint possam ser executadas de forma idêntica tanto no servidor central em Linux como nas estações offline do laboratório em Windows.

Plaintext

- NanopaintHub (Solução Principal)
- │
- ├── 📁 1.Core (Regras de Negócio Puras — Camadas Submetidas a Revisão Estrita)
- │ ├── 📁 NanopaintHub.Domain/ (Entidades: Product, TestBatch, DocumentTemplate, ProductCategory, Supplier, SupplierComponent)
- │ └── 📁 NanopaintHub.Application/ (Contratos: IProductRepository, ITemplateRepository, IDocumentGeneratorService, IUnitOfWork. UseCases e DTOs)
- │
- ├── 📁 2.Infrastructure (Músculos e Detalhes Tecnológicos Externos)
- │ ├── 📁 NanopaintHub.Persistence/ (EF Core configurado exclusivamente para o PostgreSQL do Servidor Central Corporativo)
- │ ├── 📁 NanopaintHub.LocalData/ (Configuração, Migrações do SQLite + SQLCipher, Repositórios Atómicos e classe UnitOfWork)
- │ └── 📁 NanopaintHub.Shared/ (Injeção: PuppeteerSharp local + Handlebars.NET para Windows desktop vs HTTP Gotenberg em Docker Linux)

- │
- ├── 📁 3.API (Orquestrador de Sincronismo e Zelador da Nuvem)
- │ └── 📁 NanopaintHub.WebAPI/ (API REST .NET Core — Executada em Contentor Docker no Servidor Central da Fábrica)
- │
- └── 📁 4.Clients (A Camada de Apresentação e Interação com o Utilizador)
- └── 📁 NanopaintHub.WinUIClient/ (Frontend WinUI 3 Nativo — Windows App SDK para as Estações e Portáteis Industriais)
- ├── 📁 Services/ (Network Monitor via Win32 APIs, Sincronismo em Background, Gestor de Cache JWT)
- ├── 📁 ViewModels/ (Lógica de Interação de Ecrãs baseada no CommunityToolkit.Mvvm)
- └── 📁 Views/ (Janelas XAML nativas com efeitos Fluent Design, Mica, Acrylic e controlos WebView2)

2. Estrutura de Diretórios Físicos no Disco (Ambiente Windows do Laboratório)

As pastas no sistema operativo foram mapeadas para garantir isolamento de ficheiros críticos e, simultaneamente, disponibilizar pontos de contacto *No-Code* de fácil acesso para o departamento de marketing e softwares analíticos de terceiros.

Plaintext

- 📁 C:\Users\<Utilizador>\AppData\Local\NanopaintLIMS\
- │
- ├── 📁 Browser\
- │ └── 📁 Chromium\ (Diretório embutido estável onde o binário do Chromium gerido pelo Puppeteer reside)
- │
- ├── 📁 Database\
- │ └── 📄 lims_local.db (Ficheiro SQLite local encriptado via SQLCipher — Gerado por Fluent API)
- │
- └── 📁 Logs\
- └── 📄 lims_offline_log.txt (Registo contínuo de auditoria industrial, falhas de rede e telemetria local)
-
- 📁 C:\Nanopaint\
- │

- ├── 📁 Entrada_Graficos\ (Pasta monitorizada por Background Services onde as máquinas analíticas cospem os gráficos .png)
- │ └── 📄 lote_2026_A_curva.png
- │
- ├── 📁 Marketing\
- │ └── 📁 Templates\ (Diretório de drop de ficheiros onde o marketing coloca os papéis timbrados em formato PDF vetorial)
- │ ├── 📄 timbre_tds_cabecalho.pdf
- │ └── 📄 timbre_tds_rodape.pdf
- │
- └── 📁 TDS_Geradas\ (Destino final onde as Fichas Técnicas e de Segurança são geradas em PDF final selado)
- └── 📄 TDS_InkPR02NP_Lote_2026_A.pdf

PARTE 2: MATRIZ DE ESPECIFICAÇÃO TÉCNICA E ENGENHARIA DE COMPONENTES

1. Filosofia "Modo Utilitário Local Livre" vs Sincronismo JWT

O Nanopaint LIMS opera sob um conceito de soberania pragmática: o software comporta-se localmente como um utilitário de produtividade de acesso livre.

- **Fluxo Livre Offline (Operador):** O técnico de laboratório abre a aplicação WinUI 3 sem qualquer barreira de login. Ele pode criar produtos na cache local, preencher dados de ensaios, anexar gráficos e emitir documentos oficiais (TDS/SDS) perfeitamente formatados. O sistema regista estes dados no SQLite local com o status LocalOnly. Latência zero, risco zero de paragem de fábrica por falhas de rede.
- **Fluxo de Sincronismo e Administração (Modo Conectado):** As restrições de segurança atuam exclusivamente nas portas da API Corporativa central. Para descarregar dados para o PostgreSQL corporativo ou aceder ao painel No-Code de Edição de Templates, o utilizador insere as credenciais, que são validadas pela Web API central via ASP.NET Core Identity. O servidor devolve um token JWT seguro.
- **O "Modo Viagem" (Portáteis da Engenharia):** O token JWT obtido durante a estadia na fábrica é armazenado de forma protegida na base de dados SQLite local. Quando o engenheiro viaja para um cliente ou auditor externo, o código local da aplicação WinUI 3 executa uma **Validação Matemática Offline** das *claims* e da data de expiração do JWT. Caso o token continue válido no tempo, a aplicação liberta os privilégios administrativos (como a edição de templates HTML locais) mesmo estando completamente isolada da internet.

2. Pipeline Híbrido de Geração de Documentos (Handlebars.NET +

Puppeteer + PdfSharpCore)

A geração de PDFs abandona qualquer dependência de ficheiros de texto legados (.docx) ou automação de Office, correndo de forma nativa na infraestrutura em duas camadas isoladas:

Plaintext

- [DADOS PUROS JSON] ➔ [DESSERIALIZAÇÃO EM DICIONÁRIOS] ➔ [COMPILAÇÃO HANDLEBARS.NET] ➔ [MOTOR PUPPETEER SHARP] ➔ [CAMADA 1: PDF TEXTO] ➔ [CAMADA 2: FUSÃO PDFSHARPCORE COM TIMBRE] ➔ [PDF FINAL SELADO C:\NANOPAINT]

1. **Desserialização Atómica:** O motor lê os blocos dinâmicos do domínio do produto (FeaturesJson, LiquidPropertiesJson, MechanicalPropertiesJson) e converte-os através do JsonSerializer em coleções nativas C# (List<string> e Dictionary<string, string>).
2. **Furação Semântica via Handlebars.NET:** O HTML desenhado pelo marketing (armazenado no DocumentTemplate) é compilado dinamicamente pelo Handlebars. O motor injeta os dicionários de propriedades executando loops estruturados ({{#each}}) e validações lógicas ({{#if}}) diretamente no texto do template, mantendo o código C# 100% isolado do design visual.
3. **Camada 1 (Renderização de Texto):** O Puppeteer Sharp aciona uma instância invisível e embutida do Chromium, lê o HTML gerado e imprime o documento bruto em formato PDF A4, aplicando com precisão milimétrica as margens físicas parametrizadas no banco (TopMargin, BottomMargin). Isto garante que o texto do laboratório nunca colida com os cabeçalhos do marketing.
4. **Camada 2 (Fusão de Arte Vetorial):** O PdfSharpCore captura o PDF de texto gerado pelo Puppeteer e, através de um ciclo de manipulação de matrizes de desenho, funde em plano de fundo os cabeçalhos e rodapés vetoriais em PDF fornecidos pelo marketing (timbre_tds_cabecalho.pdf), gravando o output final perfeitamente selado no disco.

3. Padrão Transacional Unit of Work (Persistência Sem Auto-Commit)

Para garantir a máxima integridade dos dados locais e remover o risco de corrupção do SQLite em caso de quebra abrupta de energia nas estações de trabalho, o sistema implementa o padrão *Unit of Work*.

- Os repositórios (ProductRepository e TemplateRepository) atuam de forma isolada e estúpida: eles apenas preparam, adicionam ou modificam os estados das entidades na memória RAM mapeada pelo contexto do Entity Framework Core. Os métodos de escrita (AddAsync, UpdateAsync, DeleteAsync) **foram limpos de qualquer chamada direta** a SaveChangesAsync().
- O Caso de Uso (Maestro) detém o controlo absoluto do ciclo transacional. Só após garantir que todas as etapas do negócio terminaram com sucesso é que o Use Case

evoca o IUnitOfWork.CommitAsync(), descarregando uma transação única, atómica e isolada diretamente no ficheiro .db físico.

PARTE 3: MODELO LÓGICO DA BASE DE DADOS (CORE EM HARMONIA)

A modelagem de dados é estritamente partilhada através do Entity Framework Core, aplicando regras rigorosas de integridade referencial via Fluent API no método OnModelCreating do contexto:

C#

```
// Configuração de Integridade e Proteção de Dados Industriais
modelBuilder.Entity<Product>()
    .HasOne<ProductCategory>()
    .WithMany()
    .HasForeignKey(p => p.CategoryId)
    .OnDelete(DeleteBehavior.Restrict); // Bloqueia a eliminação acidental de Categorias com produtos ativos
```

Máquina de Estados de Sincronismo (sync_status)

O ciclo de vida de persistência de dados é governado pelo estado mecânico da coluna SyncStatus nas tabelas operacionais:

1. **LocalOnly (Valor Default):** Atribuído automaticamente no Modo Utilitário Livre. Os dados residem exclusivamente no SQLite local e podem ser alterados ou apagados sem qualquer validação de rede.
2. **PendingInsert / PendingUpdate:** Ativado quando o operador ou administrador, com o token JWT válido e sessão ativa, solicita a sincronização dos dados com a nuvem. O Background Service tranca o registo para edição local e coloca-o na fila de envio paginado (*Chunking*).
3. **Synced:** Harmonia e consistência global. A Web API central processou os dados, validou os limites, gravou no PostgreSQL corporativo e devolveu a confirmação de sucesso. O registo volta a ficar em modo de leitura seguro no cliente.
4. **ValidationError:** Ativado quando a API central rejeita o registo devido a colisões lógicas, violações de integridade ou erros de concorrência idempotente (HTTP 409 Conflict). O registo é devolvido ao cliente e retido para retificação visual obrigatória por parte do utilizador administrador no ecrã da aplicação WinUI 3.

PARTE 4: MATRIZ DE RESILIÊNCIA E INFRAESTRUTURA DEFENSIVA (EDGE CASES)

Vetor de Risco Físico / Digital	Mitigação de Engenharia Implementada no Sistema
Isolamento de Rede / Falhas de Servidor	**Modo Utilitário Aberto:** A UI nativa em WinUI 3 não exige login para operações de rotina. Os formulários salvaguardam os dados no SQLite local e o motor Puppeteer gera PDFs na hora, mantendo a expedição de mercadorias e o laboratório ativos com latência zero.
Autenticação Fora da Intranet	**Validação JWT Matemática Offline:** O token é guardado no SQLCipher. A app desktop valida as claims criptográficas localmente através do relógio interno e do algoritmo RSA, preservando privilégios administrativos no portátil fora da fábrica.
Concorrência Local e Duplo Clique	**Mutex Global de Instância Única:** O arranque do Windows App SDK verifica a existência de um Mutex nominal no kernel do SO. Se detetada uma segunda instância da aplicação a tentar iniciar, o processo é imediatamente abortado, impedindo colisões de escrita e locks no SQLite.
Sobrecarga de Carga Única (Retoma de Rede)	**Mecanismo de Chunking Paginado:** O Background Service restringe os uploads a pacotes máximos controlados (ex: máximo de 50 registos por pedido HTTP POST). Isto mitiga a ocorrência de erros HTTP 413 (*Payload Too Large*) após longos períodos de isolamento industrial.
Vazamento de Memória RAM (Sincronismo)	**IDbContextFactory + .AsNoTracking():** Os processos de sincronização em segundo plano não reutilizam contextos. O serviço invoca uma fábrica de contextos limpa, extrai os registos sem monitorização de estado em memória e destrói o contexto de imediato após a remessa do JSON.

Colisões Cegas de Sincronismo Remoto	**Idempotência Lógica e Chaves Compostas:** O PostgreSQL central aplica uma chave única composta por numero_lote + produto_id. O primeiro utilizador a sincronizar grava com sucesso; o segundo é travado pela Web API com um HTTP 409 Conflict, forçando o status local para ValidationError.
Ataque DDoS Involuntário ao Servidor	**Exponential Backoff com Jitter:** As tentativas automáticas de reconexão do Background Service após quedas de rede não ocorrem em intervalos fixos. O sistema calcula um compasso de espera geométrico acrescido de uma variável aleatória (Jitter), espalhando os pedidos no tempo e protegendo o contentor Docker principal.

PARTE 5: ROADMAP DE IMPLEMENTAÇÃO E EXECUÇÃO ATUALIZADO

🗓️ FASE 1: O Utilitário Soberano Windows (Persistência Offline & Domínio) — [100% CONCLUÍDA]

- Modelação completa do Domínio industrial de produtos com estruturas JSON dinâmicas.
- Contratos e Use Cases de aplicação estruturados em lógica puramente abstrata.
- Criação do projeto de persistência local, Fluent API, Migrations e geração física do ficheiro nanopaint.db.
- Implementação do padrão *Unit of Work* puro nos repositórios locais para gravação atómica.
- Integração do motor Puppeteer Sharp local acoplado ao Handlebars.NET para leitura automática de JSON e renderização perfeita em PDF.

🗓️ FASE 2: Interface Gráfica WinUI 3, API Corporativa e Segurança JWT — [EM EXECUÇÃO]

- Criação da Web API em .NET Core acoplada ao ambiente Docker e base de dados PostgreSQL corporativa.
- Configuração do ASP.NET Core Identity no servidor central com emissão de chaves criptográficas JWT expiráveis.
- Desenvolvimento do projeto **NanopaintHub.WinUIClient** e desenho das janelas Fluent Design (Mica, XAML) para o laboratório.
- Implementação do ecrã de Login para administradores, mecanismo de captura, encriptação no SQLCipher e validação matemática offline do JWT (Modo Viagem).

- Libertação do componente de edição visual de templates (WebView2 + Quill) trancado sob a validação do token JWT de administrador.
- Ativação do Background Service nativo do Windows App SDK dotado de Network Monitor, Chunking e Exponential Backoff para sincronização de dados.

🗓️ FASE 3: Automação Avançada, Gotenberg Server e Cloud Híbrida — [FUTURO]

- Programação do motor local de compilação de riscos SDS (agregação automática de frases H/P e pictogramas de matérias-primas de fornecedores).
- Implementação das regras de validação cruzada de limiares TDS em runtime com injeção automática de selos de conformidade digitais.
- Configuração do contentor Gotenberg no servidor Linux corporativo para renderização de PDFs em lote a pedido do CRM Web corporativo do Luís.
- Deploy definitivo da Web API em produção no MonsterAsp.net e desvio de ficheiros de arquivo histórico pesados para os buckets de custo zero de egresso do Cloudflare R2.